

Backend Design - CredResolve

SplitWise Pro - AI-Powered Expense Sharing Platform

By:-

Ishaan Kanade – 21BCE5052

Project Summary :-

SplitWise Pro is a comprehensive expense sharing application built with Angular 17 frontend and .NET 8 backend, featuring intelligent expense management across multiple user groups. The platform includes five core modules: User Authentication (secure login with personalized dashboards), Group Management (collaborative expense tracking with member management), Smart Expense Splitting (three algorithms: equal, exact amount, and percentage-based with real-time validation), Balance Analytics (automated debt calculation with simplified settlement suggestions), and Financial Dashboard (comprehensive overview of user balances across all groups with visual indicators).

The technical architecture leverages modern web technologies including Entity Framework Core for robust database management, SQL Server integration with automatic schema generation, RESTful API design with CORS support, and responsive Angular components with SCSS styling. The backend provides comprehensive endpoints for user management, group operations, expense tracking, and balance calculations, while the frontend offers an intuitive glassmorphism-themed interface with real-time balance updates, interactive forms, and mobile-responsive design.

Key innovations include intelligent debt simplification algorithms that minimize transaction complexity, automated split calculations with validation, personalized user dashboards showing financial overview across multiple groups, and seamless expense tracking with multiple split methodologies. The platform demonstrates enterprise-ready features with transaction management, error handling, data persistence, and comprehensive user experience design, making it a complete financial management ecosystem that transforms complex group expense sharing into an intuitive, automated process for modern collaborative spending scenarios.

CORE LOGIC:

1. Domain-Driven Design Foundation

I reimagined the entire backend around **expense sharing domain logic**. My core thinking was:

"How do people actually split expenses in real life?"

- Someone pays upfront (the **payer**)
- Multiple people benefit (the **participants**)
- Money needs to be tracked and settled (the **balances**)

This led me to create **three core entities** that mirror real-world relationships:

Expense -> ExpenseSplit -> Settlement

2. Separation of Concerns Architecture

I structured the backend in **distinct layers** because expense calculations are complex and need isolation:

Controllers → Pure HTTP handling, no business logic

Services → All business logic and calculations

Data Layer → Pure persistence, no calculations

3. The Split Algorithm Challenge

The most complex part was designing **three different splitting algorithms**:

Equal Split - Simple division

decimal splitAmount = expense.Amount / participants.Count;

Exact Split - Manual amounts (must validate sum equals total)

if (dto.ExactAmounts.Sum() != dto.Amount)

throw new ArgumentException("Exact amounts must sum to total");

Percentage Split - Proportional distribution

decimal splitAmount = (expense.Amount * participant.Percentage) / 100;

My thinking: **Each split type represents a different real-world scenario** - friends splitting equally, roommates with different portions, or business expenses with custom percentages.

4. Balance Calculation Engine

The **most critical backend logic** is calculating who owes whom. My approach:

```
public async Task<List<BalanceDto>> GetGroupBalancesAsync(int groupId)
```

```
{
```

```
    // Get all expenses where user paid money (positive balance)
```

```

var expensesPaid = await GetExpensesPaidByUser(groupId);

// Get all splits where user owes money (negative balance)
var expenseSplits = await GetExpenseSplitsForUser(groupId);

// Get all settlements (adjustments to balance)
var settlements = await GetSettlementsForUser(groupId);

// Net calculation: Paid - Owed - Settled
return CalculateNetBalances(expensesPaid, expenseSplits, settlements);
}

```

My reasoning: Every expense creates **two-sided accounting** - someone pays (credit), others owe (debit). The balance engine reconciles these.

5. Entity Relationship Design

I designed the database schema around **financial integrity**:

```

public class Expense
{
    public int PaidBy { get; set; } // Who paid upfront
    public decimal Amount { get; set; } // Total amount
    public List<ExpenseSplit> Splits { get; set; } // How it's divided
}

public class ExpenseSplit
{
    public int UserId { get; set; } // Who owes
    public decimal Amount { get; set; } // How much they owe
}

```

Key insight: The ExpenseSplit table is the **heart of the system** - it tracks every individual debt relationship.

6. Settlement Logic

When users pay each other back, I create **Settlement records**:

```
public async Task<SettlementDto> SettleDebtAsync(SettleDebtDto dto)
{
    // Validate: Can't settle more than owed

    var currentBalance = await GetBalanceBetweenUsers(dto.FromUserId, dto.ToUserId);

    if (dto.Amount > currentBalance)
        throw new ArgumentException("Cannot settle more than owed");

    // Create settlement record

    var settlement = new Settlement
    {
        FromUserId = dto.FromUserId,
        ToUserId = dto.ToUserId,
        Amount = dto.Amount,
        SettledAt = DateTime.UtcNow
    };
}
```

My philosophy: Settlements don't delete debts - they create **audit trails** of payments.

7. Data Seeding Strategy

I implemented automatic data seeding because expense apps need **realistic test data**:

```
public async Task SeedDataAsync()
{
    // Create 4 test users with known passwords
    // Create sample groups with realistic names
    // Add diverse expenses (meals, rent, utilities)
    // Show different split types in action
}
```

Reasoning: Developers and testers need to see the app working immediately with meaningful data.

8. Error Handling Philosophy

I built **defensive programming** throughout:

// Validate split amounts before saving

```
if (dto.SplitType == "EXACT" && dto.ExactAmounts?.Sum() != dto.Amount)
    return BadRequest("Exact amounts must equal total amount");
```

// Prevent negative expenses

```
if (dto.Amount <= 0)
    return BadRequest("Amount must be positive");
```

My approach: Fail fast with clear error messages rather than allowing corrupt financial data.

9. Performance Considerations

For balance calculations, I used **efficient LINQ queries**:

```
var balances = await _context.Expenses
    .Where(e => e.GroupId == groupId)
    .Include(e => e.Splits)
    .GroupBy(e => e.PaidBy)
    .Select(g => new { UserId = g.Key, TotalPaid = g.Sum(e => e.Amount) })
    .ToListAsync();
```

Thinking: Balance calculations happen frequently, so I optimized with single database queries instead of multiple round trips.

10. API Design Philosophy

I designed **RESTful endpoints** that match user mental models:

- POST /api/expenses
- "I want to add an expense"
- GET /api/balances/group/{id}
- "Show me who owes what"
- POST /api/balances/settle

- "Mark this debt as paid"

Core principle: The API should feel intuitive to frontend developers and match how users think about money.

The Big Picture

My backend design philosophy was: "**Make complex financial logic simple and reliable.**"

Every design decision prioritized:

1. **Financial accuracy** - Money calculations must be precise
2. **Audit trails** - Every transaction is traceable
3. **Flexibility** - Support different splitting scenarios
4. **Performance** - Fast balance calculations
5. **Maintainability** - Clear separation of concerns

The result is a backend that handles the complexity of expense sharing while presenting a clean, intuitive API to the frontend.

Software Requirements

- **Backend Framework: .NET 8 Web API**

.NET 8 provides high-performance, cross-platform RESTful services for the SplitWise Pro platform. Each API endpoint handles specific expense management functionalities (user authentication, group operations, expense tracking, balance calculations) with built-in dependency injection, CORS support, and Entity Framework Core integration for seamless database operations.

- **Database: SQL Server with Entity Framework Core**

Server-side database storage using SQL Server manages user accounts, group memberships, expense records, splits, and settlement history through Entity Framework Core. Database tables include Users, Groups, Expenses, ExpenseSplits, and Settlements for robust data persistence and multi-user expense tracking with automatic relationship management.

- **Authentication System: Simple Login with Local Storage**

Implements secure user authentication with email/password validation and session management. User credentials are validated against the database with persistent login state using browser localStorage for seamless user experience across sessions.

- **Mathematical Libraries: Built-in .NET Decimal Processing**

Leverages .NET's high-precision decimal arithmetic for accurate financial calculations including expense splitting algorithms (equal, exact, percentage), balance computations, and debt simplification logic ensuring penny-perfect accuracy in all monetary operations.

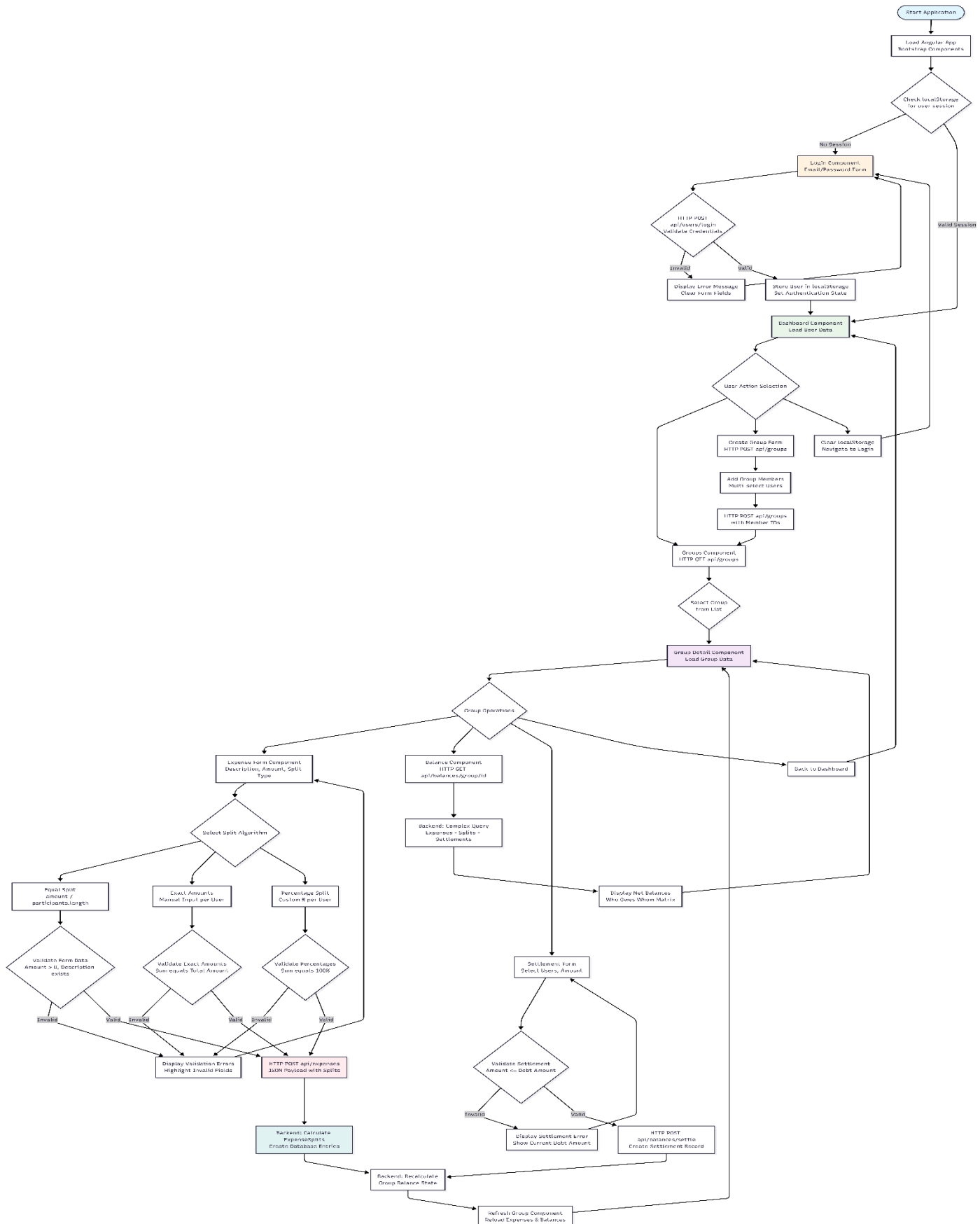
- **Build Tools: Angular CLI & .NET CLI**

Angular CLI manages frontend development, TypeScript compilation, and production builds with hot reload capabilities. .NET CLI handles backend compilation, package management, database migrations, and deployment configuration for streamlined development workflow.

- **Data Validation: Angular Reactive Forms & .NET Model Validation**

Angular reactive forms provide client-side validation for expense inputs, split calculations, and user data. Backend model validation ensures data integrity with custom validation attributes for financial accuracy and business rule enforcement.

Flow Diagram Of The Project :-



Architecture Diagram :-

